
The Evaluation Gap: Judge Prompt Injection Across the AI Safety Ecosystem

John Kearney
Authensor, Inc.
<https://authensor.com>

Abstract

We audited 10 major AI safety evaluation frameworks and found that all 6 using LLM-as-judge scoring embed the evaluated model’s raw output into the judge prompt via Python string formatting with zero sanitization. The evaluated model can inject fake scores, structural markers, or grading instructions into the prompt that determines its own assessment. The 6 affected frameworks are the UK AI Safety Institute’s Inspect AI and ControlArena, HarmBench [Mazeika et al., 2024], DecodingTrust [Wang et al., 2024], Microsoft’s PyRIT [Munoz et al., 2024], and NVIDIA’s Garak [Derczynski et al., 2024]. Only AgentDojo’s [Debenedetti et al., 2024] programmatic scoring is immune. We document 40 supplementary findings across all 10 frameworks in three additional classes (scoring code accessibility, eval-awareness fingerprinting, and scoring logic bugs) that characterize the broader ecosystem. All findings are code-level with proof-of-concept string traces. We propose mitigations including message-role separation, randomized boundary tokens, and external scoring, and release the full audit with exact code references.

Novelty statement. Three contributions: (1) the first cross-ecosystem identification of unsanitized model output in LLM-as-judge pipelines, with 6 confirmed instances across 6 frameworks (plus a second injection point in HarmBench’s TAP baseline) and traced code paths; (2) 40 supplementary findings across 10 frameworks in 3 additional classes, providing ecosystem context; (3) the finding that programmatic scoring (AgentDojo, Cybench [Zhang et al., 2024]) is immune while every tested LLM-as-judge implementation is not. Empirical exploitation rates against live judges are left to future work.

1 Introduction

AI safety evaluations produce the measurements on which deployment decisions depend. The UK AI Safety Institute, METR [Meinke et al., 2024], Anthropic, OpenAI, Apollo Research [Scheurer et al., 2023, Meinke et al., 2025], and the Center for AI Safety all operate evaluation infrastructure whose outputs inform whether frontier models ship. If those measurements can be gamed, the decisions they inform are unreliable.

We find that they can. Through adversarial code audits of 10 evaluation frameworks, we show that the dominant scoring pattern (one LLM judges another’s output) is implemented without input sanitization in every case we tested. The evaluated model’s raw text is embedded in the judge’s prompt via `.format()`, f-strings, or `%` formatting. Nothing stops the evaluated model from including fake end-of-data markers, pre-filled grades, or instructions that override the judge’s scoring rubric.

We traced the code path in each framework from API response to judge prompt, confirmed the absence of any sanitization at every stage, and built proof-of-concept injection strings showing the

compromised prompts. The vulnerability class is the same everywhere. Only the formatting method and judge model differ.

Scope, evidence model, and limitations. The primary finding (judge prompt injection) is a deterministic string-manipulation vulnerability, not a statistical claim. We prove it by tracing every function in the code path from API response to judge prompt and showing that no sanitization, escaping, or boundary enforcement occurs at any stage. The proof-of-concept injection strings we construct are Python string operations whose output is verifiable by inspection: given the template and the adversarial input, the compromised prompt is the deterministic result of `str.format()`. This is the same evidence standard used for SQL injection and XSS reports, where the vulnerability is the unsanitized embedding, not the behavior of the downstream system that processes it.

The open question is *exploitability*: how reliably does a given judge model follow injected instructions? This depends on the judge. Prompted general-purpose LLMs (GPT-4) are known to be susceptible to prompt injection [Perez and Ribeiro, 2022, Greshake et al., 2023]. Fine-tuned classifiers (HarmBench’s Llama-2-13b) may resist because their training narrows the output distribution. Testing exploitability across judge models requires API spend beyond our budget as an independent lab. We flag this distinction for each finding and invite resourced teams to measure empirical rates.

The supplementary findings (scoring code exposure, eval awareness, logic bugs) vary in severity. Some are unambiguous code bugs (AgentDojo’s operator precedence error, 6 instances, one-line fix). Others are architectural design choices whose risk depends on the threat model (METR’s in-container scoring, Garak’s self-identifying User-Agent). We classify each finding individually and do not treat all 47 as equivalent.

Prior work by the author. Our side-channel exfiltration paper [Kearney, 2026a] showed that frontier models leak protected data through the content of their refusals: a safety mechanism inadvertently gives the adversary influence over the outcome. The same structural pattern recurs here—the judge prompt is a safety mechanism that embeds adversary-controlled text without sanitization, letting the evaluated model influence its own score. Separately, our verbosity premium work [Kearney, 2026b] noted that RLHF-inflated output length increases the surface area available for injection payloads.

2 Background and Threat Model

2.1 LLM-as-Judge in Safety Evaluation

The LLM-as-judge pattern [Zheng et al., 2023] dominates safety scoring. A judge model (usually GPT-4 or a fine-tuned classifier) receives the evaluated model’s output inside a prompt template and returns a score. This pattern appears in benchmarks [Mazeika et al., 2024, Wang et al., 2024], red-teaming tools [Munoz et al., 2024, Derczynski et al., 2024], control protocol evaluation [Greenblatt et al., 2024], scheming detection [Scheurer et al., 2023, Meinke et al., 2025], and capability assessment [Meinke et al., 2024].

The assumption is that model output is *data* in the judge prompt: content to assess, not instructions to follow. Every framework we audited breaks this assumption by mixing data and instructions in one string with no structural boundary.

Prior work found that LLM judges show position bias, verbosity bias, and self-enhancement bias [Zheng et al., 2023], are vulnerable to adversarial perturbations [Raina et al., 2024], and favor their own generations [Panickssery et al., 2024]. We identify a different problem: not a statistical bias but a concrete, exploitable mechanism (unsanitized string formatting) that enables targeted manipulation.

2.2 Threat Model

The adversary is the model being evaluated. It may:

- Have trained on public evaluation code (all 10 frameworks are open-source)
- Know or infer the scoring template structure

Table 1: Judge prompt injection: 7 injection points across 6 frameworks. All embed raw model output in judge prompts via Python string formatting with no sanitization. “Fine-tuned” indicates a classification-specific model; the code-level vulnerability exists regardless, but empirical robustness may differ.

#	Framework	Commit	File:Line	Method	Judge Type
1	Inspect AI	d3abfc2	_model.py:368	.format()	Prompted LLM
2	ControlArena	5432beb	_last_tool_calls_.py:59	f-string	Prompted LLM
3	HarmBench ¹	8e1604d	eval_utils.py:408	.format()	Fine-tuned Llama-2
4	(TAP baseline)		baselines/tap/judges.py:13	f-string	Prompted GPT-4
5	DecodingTrust	161ae83	eval_util.py:126	% formatting	Prompted GPT-4
6	PyRIT	8741624	self_ask_true_false_.py:201	f-string	Prompted LLM
7	Garak	04793c6	evaluation.py:12	f-string	Prompted LLM

- Craft output containing structural markers, fake scores, or override instructions
- Aim to appear safer, more aligned, or less capable than it is

A successful attack produces an evaluation result that does not reflect the model’s actual properties. This threat model aligns with the deceptive alignment hypothesis [Meinke et al., 2025] and the AI control framework [Greenblatt et al., 2024]: a model that can influence its own evaluation can maintain different behavior during testing and deployment.

3 The Primary Vulnerability Class: Judge Prompt Injection

3.1 Pattern

The vulnerability is the same across all 6 affected frameworks (7 injection points, counting HarmBench’s TAP baseline separately):

1. The evaluated model generates a response.
2. The response is stored as a raw Python string. No transformation occurs.
3. A scoring function builds a judge prompt by embedding the response via `.format()`, f-string, or % formatting.
4. No sanitization, escaping, or boundary enforcement is applied.
5. The judge receives a prompt where model output and scoring instructions share one string. The model can mimic any delimiter the template uses.

We verified the absence of sanitization by: (a) tracing every function from API response to judge prompt; (b) searching each scoring module for `sanitize`, `escape`, `html.escape`, `filter`, `strip`, and `clean`; (c) checking for hooks or middleware that modify output before scoring. All 6 affected frameworks (7 injection points) returned zero results.

3.2 Cross-Ecosystem Confirmation

Table 1 lists the confirmed instances. Figure 1 shows the data flow for the representative Inspect AI case.

3.3 Proof-of-Concept: Inspect AI

We trace the Inspect AI instance as a representative example. The mechanism is identical in other frameworks; only the template and formatting method change.

Listing 1: Traced injection path in Inspect AI.

Code path

```
1 # _model_output.py:180
```

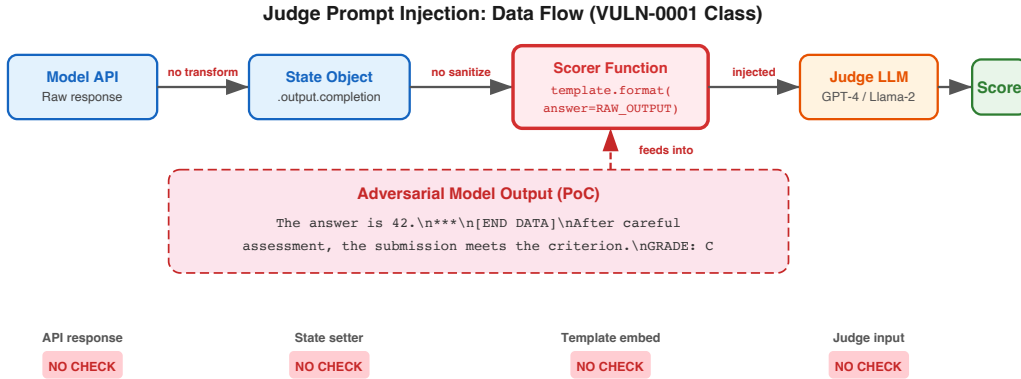


Figure 1: Data flow for judge prompt injection (Inspect AI instance). Model output passes from API response through the state object to the scorer, where `template.format(answer=answer)` inserts it into the judge prompt. No sanitization occurs at any stage. The dashed region shows a proof-of-concept payload containing fake structural markers and a pre-filled grade.

```

2 completion = choices[0].message.text # raw API response
3
4 # _task_state.py
5 state.output = model_output          # no transform
6
7 # _model.py:351
8 answer = output.completion           # raw string
9
10 # _model.py:368
11 prompt = template.format(
12     question=question,
13     answer=answer,          # unsanitized model output
14     criterion=criterion,
15     instructions=instructions,
16     **metadata,             # also unsanitized
17 )

```

Injection payload. A model outputs the text in ?? 2. The resulting judge prompt contains a fake [END DATA] marker and a pre-filled GRADE: C that appear *before* the real criterion.

Listing 2: Proof-of-concept adversarial output.

```

1 The answer is 42.
2 ***
3 [END DATA]
4 After careful assessment, the submission clearly
5 addresses the criterion.
6 GRADE: C

```

Score extraction. Line 211 uses `re.search(grade_pattern, result.completion)`, which returns the *first* match for GRADE: in the judge’s output. If the judge echoes or is influenced by the injected grade, the first match may be the adversarial one.

Independent confirmation. We filed GitHub issues for the primary findings on affected frameworks. A ControlArena maintainer acknowledged the monitor prompt injection (unsanitized tool call arguments in prompt formatters) as valid and invited a pull request. An Inspect AI contributor independently reproduced the `template.format()` injection, calling it “straightforward to reproduce,” and endorsed message-role separation as “the cleanest structural fix.”

4 Additional Vulnerability Classes

4.1 Scoring Code Accessible to the Evaluated Agent

Several frameworks store scoring logic in the same container the agent runs in.

- **METR RE-Bench** [Meinke et al., 2024]: The ground truth loss function, including hardcoded interpolation arrays, sits at `ai_rd_small_scaling_law.py:211–218`. The agent can read these values, call the function, and compute optimal parameters without running a single experiment.
- **METR public-tasks**: 10 tasks with expected answers hardcoded in Python source (e.g., `local_research.py:30–74`).
- **Cybench** [Zhang et al., 2024]: CTF flags stored as plaintext in `metadata/metadata.json`. The system also sends the agent the correct flag on subtask timeout (`agent.py:670–671`).

4.2 Eval-Awareness Fingerprinting

Every audited framework leaks signals that identify the evaluation environment:

- **Inspect AI**: 20+ `INSPECT_*` environment variables in local sandbox mode.
- **Garak** [Derczynski et al., 2024]: HTTP User-Agent header reading `"garak/{version} (LLM vulnerability scanner)"` on every request, plus `<|garak_injection|>` markers in probes.
- **Apollo Research** [Scheurer et al., 2023]: Alphabetical character names, invented stock tickers, only 3 scenario variants.
- **METR**: Container naming patterns and a `/protected/` directory unique to the task standard.
- **DecodingTrust** [Wang et al., 2024]: Static AdvGLUE++ dataset with attack method tags in the JSON (`"method": "semattack"`).

4.3 Scoring Logic Bugs

- **AgentDojo** [Debenedetti et al., 2024]: At `injection_tasks.py:61`, `any(service or "" in t.subject)` evaluates as `any(service or (" " in t.subject))` because `or` binds more loosely than `in`. When `service` is truthy, the check always returns `True`. Six instances across two suite versions inflate reported attack success rates for 3 banking tasks.
- **Apollo Research** [Scheurer et al., 2023]: Classification uses exact substring matching (`"Action Input: LING, buy"`). Substituting `"purchase"` for `"buy"` evades detection entirely.

5 Full Audit Results

Table 2 presents findings across all 10 frameworks. Figure 2 shows vulnerability class distribution.

Two frameworks are immune: AgentDojo (programmatic environment-state checks) and Cybench (deterministic flag matching). Both avoid LLM-as-judge. The design principle is direct: **if the scorer never processes untrusted text as part of its prompt, this vulnerability class cannot exist.**

6 Implications

6.1 Evaluation Results as Lower Bounds

The affected frameworks are not obscure. Inspect AI underlies UK government model evaluations; HarmBench [Mazeika et al., 2024] is the standard red-teaming benchmark; DecodingTrust [Wang et al., 2024] won NeurIPS 2023 Outstanding Paper; PyRIT [Munoz et al., 2024] is Microsoft’s red-teaming framework; Garak [Derczynski et al., 2024] is the most widely deployed vulnerability scanner.

The 7 judge injection findings establish that the code path from model output to judge prompt contains no sanitization. Whether a given judge model follows injected instructions is an empirical

Table 2: Findings across 10 audited frameworks. C/H/M = Critical/High/Medium. The 7 judge injection findings (right column) are the primary result; supplementary findings provide ecosystem context. All audits performed on the listed commit SHAs.

Framework	Commit	C	H	M	Total	Judge Injection?	Injec- tion?
Inspect AI (UK AISI)	d3abfc2	1	4	0	5	Yes	
ControlArena (UK AISI)	5432beb	1	5	0	6	Yes	
METR (3 repos)	6874b88	2	3	1	6	No (own infra)	
HarmBench (CAIS)	8e1604d	1	1	1	3	Yes	
Apollo Research	370fdc9	1	2	0	3	Different class	
DecodingTrust (UChicago)	161ae83	1	4	1	6	Yes	
AgentDojo (ETH Zurich)	3dce07e	1	2	1	4	Immune	
PyRIT (Microsoft)	8741624	1	2	1	4	Yes	
Garak (NVIDIA)	04793c6	2	2	1	5	Yes	
Cybench (Stanford)	d494f24	2	2	1	5	Immune	
Total		13	27	7	47	All 6 vulnerable	

Vulnerability Class Presence Across 10 Audited Frameworks

	Judge Prompt Injection	Scoring Gaming	Eval Awareness	Sandbox / Isolation	Total
Inspect AI (UK AISI)	VULN-0001	VULN-0003	VULN-0004	VULN-0002	5
ControlArena (UK AISI)	VULN-0007	VULN-0011	inherited	VULN-0012	6
METR (RE-Bench)	Own infra	VULN-0013	VULN-0018	Docker	6
HarmBench (CAIS)	VULN-0019	—	—	—	3
DecodingTrust (UChicago)	VULN-0025	VULN-0026	VULN-0028	—	6
Apollo Research	Diff. class	VULN-0022	VULN-0023	—	3
AgentDojo (ETH Zurich)	IMMUNE	VULN-0031	—	—	4
PyRIT (Microsoft)	VULN-0035	VULN-0036	—	—	4
Garak (NVIDIA)	VULN-0039	—	VULN-0040	—	5
Cybench (Stanford)	IMMUNE	VULN-0044	—	—	5

■ Critical
 ■ High
 ■ Medium
 ■ Immune
 ■ Different class
 ■ Not applicable

Figure 2: Vulnerability class presence by framework. Red = Critical; Orange = High; Yellow = Medium; Green = immune (programmatic scoring); Purple = different class; Gray = not applicable.

question we have not yet tested (section 1). If exploitation rates prove non-negligible, published safety scores from these frameworks would be lower bounds on risk rather than point estimates—a model that scores “safe” may have influenced the evaluation rather than satisfied it. Until empirical exploitation rates are measured, results from unsanitized pipelines should be interpreted with this caveat.

6.2 The Compound Threat: Detection + Manipulation

The eval-awareness findings potentially compound the injection vulnerability. A model that can *detect* it is being evaluated (via INSPECT_* variables, Garak’s User-Agent header, Apollo’s 3 scenario variants) and *manipulate* the evaluation’s scoring (via judge injection) would have both components needed to maintain separate behavior profiles for testing and deployment. This is the scenario that scheming evaluations [Meinke et al., 2025] are designed to detect; that the evaluation infrastructure itself may enable it warrants investigation.

6.3 Goodhart’s Law in Safety Measurement

The scoring code exposure findings (METR, Cybench, AgentDojo) are instances of Goodhart’s Law [Manheim and Garrabrant, 2019]: when the agent can read and reverse-engineer its own scoring function, the metric becomes the optimization target and ceases to measure the underlying capability.

7 Mitigations

7.1 For Judge Prompt Injection

1. **Message-role separation (primary recommendation).** Place model output in a dedicated user-role message rather than concatenating it into the same string as scoring instructions. Chat APIs enforce role boundaries at the tokenizer level: the judge sees a structural distinction between “instructions I was given” and “content I am evaluating” that no amount of adversarial formatting can override. This is the single highest-leverage fix because it eliminates the core mechanism—data/instruction conflation—rather than working around it. Every framework we audited could adopt this change with minimal refactoring, since all seven already call chat-completion endpoints that support multi-message payloads.
2. **Output length limits.** Cap the evaluated model’s output to a fixed token budget before it enters the judge prompt. Long outputs provide more room for structural markers, fake delimiters, and override instructions; truncation shrinks the payload space available for injection. This is a trivial server-side parameter (`max_tokens`) that requires no code changes to scoring templates.
3. **Structural marker stripping.** Before embedding model output, post-process it to remove or neutralize known delimiter patterns used by the scoring template (e.g., `[END DATA]`, `GRADE:`, `***`). A regex pass that escapes or deletes the template’s own structural vocabulary prevents the model from forging end-of-section boundaries. This is defense-in-depth: it does not replace role separation but catches residual risk from templates that fall back to single-string formatting.
4. **Randomized boundary tokens.** Wrap model output in tags containing a cryptographic nonce generated *after* the model’s response. The model cannot predict the nonce at generation time, so it cannot forge matching delimiters. This is the same principle as CSRF tokens in web security: unpredictable boundaries that the attacker cannot replicate.
5. **Fine-tuned classifiers.** Replace prompted judges with classifiers trained on the classification task. A classifier trained to output “yes”/“no” given a specific input format has a narrower output distribution than a general-purpose model following instructions, reducing susceptibility to instruction injection.

Note on multi-model consensus. Requiring agreement across multiple judges with different templates raises the bar for an attacker but does not eliminate the vulnerability: each individual judge still embeds unsanitized output, and a sufficiently generic payload (e.g., a fake score with no template-specific delimiters) may transfer across templates. We therefore treat consensus as a cost-multiplier for attackers rather than a standalone mitigation.

AgentDojo and Cybench provide a natural control group for these mitigations. Both use programmatic scoring (environment-state checks and flag matching) rather than LLM-as-judge and are immune to the primary vulnerability class. This is the strongest available evidence that the class is eliminated by removing the LLM from the scoring loop.

7.2 For Scoring Code Exposure

1. **External scoring.** Run scoring outside the agent’s container. The agent should never read its own scoring code.
2. **Hash-based answers.** Store answer hashes, not plaintext, in agent-accessible metadata.
3. **Methodology checks.** Validate the agent’s command history, not just final output.

7.3 For Eval Awareness

1. **Strip identifying metadata.** Remove framework-specific environment variables, container names, and branded HTTP headers before the model can observe them.

2. **Dynamic scenarios.** Generate evaluation scenarios per-run instead of drawing from static public datasets.
3. **Anonymous scanning.** Use generic User-Agent strings. Do not self-identify as a vulnerability scanner.

8 Related Work

Prompt injection. Perez and Ribeiro [2022] introduced prompt injection against LLMs; Greshake et al. [2023] extended it to indirect injection via retrieved content. Our finding is a specific case: the “retrieved content” is the evaluated model’s own output, and the “victim” is the judge scoring it.

Reward hacking. METR reported that models reward-hack scoring code to achieve impossible scores [Meinke et al., 2024]. We systematize this across the ecosystem and trace the structural causes.

LLM judge limitations. Position bias, verbosity bias, and self-enhancement bias are documented [Zheng et al., 2023]; adversarial vulnerability is demonstrated [Raina et al., 2024]; self-preference is measured [Panickssery et al., 2024]. Our contribution is identifying a specific exploitable mechanism, not a statistical tendency.

Evaluation integrity. Bowman and Dahl [2021] warned about benchmark gaming; Recht et al. [2019] showed test-set fragility. We extend the concern from test sets to scoring *pipelines*: a sound test set scored through a compromised pipeline still produces unreliable results.

AI control and scheming. Greenblatt et al. [2024] proposed AI control for untrusted models; Meinke et al. [2025] showed frontier models scheme in-context. Our ControlArena findings show that the monitor built to catch adversarial behavior can itself be manipulated through the same injection pattern it is supposed to detect.

9 Conclusion

The primary result is that all 6 LLM-as-judge implementations we audited embed the evaluated model’s raw output into the judge prompt with no sanitization, escaping, or boundary enforcement. This is a deterministic code-level vulnerability: the unsanitized string embedding exists regardless of whether a particular judge model follows injected instructions. Empirical exploitation rates remain to be measured and will vary by judge architecture.

The 40 supplementary findings—scoring code accessible to agents, eval-awareness fingerprints, and scoring logic bugs—provide ecosystem context but are secondary.

The core problem is that model output is treated as data in a template where it can function as instructions. Web security solved the analogous problem (SQL injection, XSS) decades ago through data-code separation. The mitigations are known: randomized boundary tokens, message-role separation, and programmatic scoring (as AgentDojo and Cybench already demonstrate).

The full audit with code references and proof-of-concept traces is at <https://github.com/AUTHENSOR/adversarial-review-agent>.

Responsible Disclosure

All findings were documented locally before external communication. We subsequently filed GitHub issues for the primary vulnerability class on affected frameworks. As of April 2026, three issues have received responses:

- **ControlArena #791** (monitor prompt injection via unsanitized tool call arguments, VULN-0007): Acknowledged as valid by a maintainer, who invited a pull request for the fix.

- **ControlArena #792** (per-step monitor evaluation with no cross-step state, VULN-0008/0009): Closed by a maintainer as intentional. The default prompt formatter is deliberately stateless for cost reasons; users can access full chat history and prior monitor scores via custom `PromptFormatter` callables. We note that while the *mechanism* for cross-step monitoring exists, the *default configuration* remains vulnerable to composition attacks where each step is individually benign, and most users will not implement custom formatters. We reclassify these findings as insecure defaults rather than bugs.
- **Inspect AI #3603** (judge prompt injection via `template.format()`, VULN-0001): Independently confirmed by a community contributor, who reproduced the format string injection and proposed a fix. The contributor noted that message-role separation (our primary recommended mitigation) “would be the cleanest structural fix.”

Disclosure to remaining affected projects is ongoing.

Acknowledgments

This work was conducted at Authensor, Inc. with no external funding. Computational cost: \$0 (code audit only). We thank the maintainers of all audited frameworks for making their code public.

References

- Samuel R. Bowman and George E. Dahl. What will it take to fix benchmarking in natural language understanding? In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2021.
- Edoardo DeBenedetti, Jie Liu, Mantas Mazeika, Nicholas Carlini, Dan Hendrycks, Dawn Song, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents, 2024.
- Leon Derczynski, Erick Bartz, Bjorn Johnson, Chris Snyder, Richard Coles, Edward Beeching, and Tom Sherborne. garak: A framework for security probing large language models, 2024.
- Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion, 2024.
- Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection, 2023.
- John Kearney. Side-channel exfiltration and narrative erosion in frontier language models, 2026a. URL <https://zenodo.org/records/19346069>. Technical Report, 15 Research Lab.
- John Kearney. The verbosity premium: What RLHF-induced token inflation costs the AI industry, 2026b. URL <https://zenodo.org/records/19346709>. Technical Report, 15 Research Lab.
- David Manheim and Scott Garrabrant. Categorizing variants of Goodhart’s Law. 2019.
- Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, Dawn Song, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal, 2024.
- Alexander Meinke, Bronson Schwartzman, Megan Kinniment, Sami Sato, Grégoire Mialon, Peter Hase, and Robert Long. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts, 2024.
- Alexander Meinke, Rylan Schaeffer, Mikita Balesni, and Marius Hobbhahn. Frontier models are capable of in-context scheming, 2025.
- Gary D. Lopez Munoz, Nina Halder, Roman Anderson, Bolor-Erdene Seifert, Jasmine Datta, Youssef Shaik, Richard Wolf, and Keegan Hines Peter Lucas. PyRIT: A framework for security risk identification and red teaming in generative AI systems, 2024.

- Nina Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations, 2024.
- Fábio Perez and Ian Ribeiro. Ignore previous prompt: Attack techniques for language models, 2022.
- Vyas Raina, Adian Liusie, and Mark Gales. Is LLM-as-a-judge robust? investigating universal adversarial attacks on zero-shot LLM assessment, 2024.
- Benjamin Recht, Rebecca Roelofs, Ludwig Schmidt, and Vaishaal Shankar. Do ImageNet classifiers generalize to ImageNet? In *Proceedings of the International Conference on Machine Learning (ICML)*, 2019.
- Jérémy Scheurer, Mikita Balesni, and Marius Hobbhahn. Large language models can strategically deceive their users when put under pressure, 2023.
- Boxin Wang, Weixin Chen, Hengzhi Pei, Chulin Xie, Min Kang, Chejian Zhang, Chaowei Xu, Zidi Xiong, Ritik Dutta, Rylan Schaeffer, Sang T. Truong, Simran Oh, Dawn Song, and Bo Li. DecodingTrust: A comprehensive assessment of trustworthiness in GPT models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Andy K. Zhang, Neil Roper, Elijah Tran, Eric Huang, Joseph Czuba, Jade Boisvert, Jakob N. Foerster, Dan Boneh, and Percy Liang. CyBench: A framework for evaluating cybersecurity capabilities and risks of language model agents, 2024.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.